

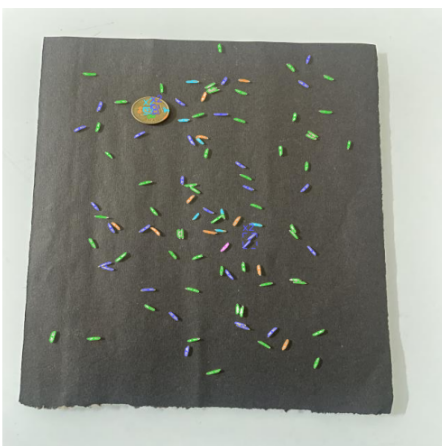
基于尺度自标定与形状先验校正的粘连米粒计数方法

数字图像处理课程作业

摘要：针对二值化后统计连通域计数时，相互接触的米粒被并作一粒、米粒越密漏计越多的问题，提出一种以图像自身尺度为基准的粘连米粒计数方法。该方法先由图像量出单粒米的面积与形状，据此判定哪些连通域为粘连块并控制注水分割的种子数量，再按面积与形状复核切分结果，并剔除硬币、木框等非米目标。在真实照片、自制可控粘连合成图与低分辨率图像三组数据上，平均绝对误差分别为 4.73、1.77 与 1.93 粒，优于五种经典对比方法；粘连率为 80% 时，误差约为直接统计连通域的十一分之一。此外，以视觉大模型 SAM 3 的输出作为伪标签，蒸馏出参数量 27 万的轻量网络，其总体误差低于上述几何方法，而模型体积约为 SAM 3 的三千分之一。文中同时记录了经实验否定的若干设想与尚未解决的问题。

关键词：米粒计数；粘连目标分割；标记控制注水分割；尺度自标定；知识蒸馏

真实照片：数出 99 粒（真值 98 粒）



合成图，粘连率 80%：数出 64 粒（真值 64 粒）

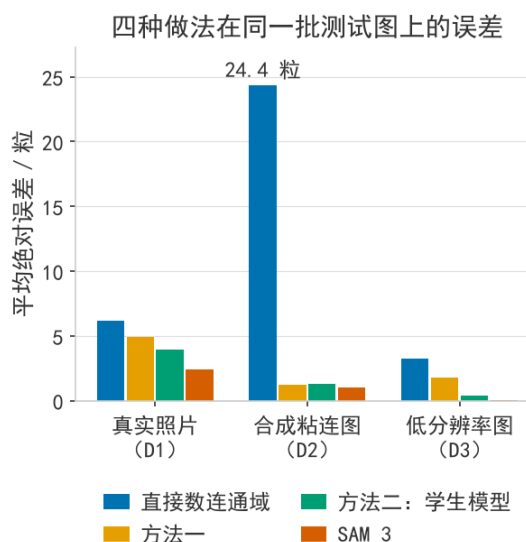
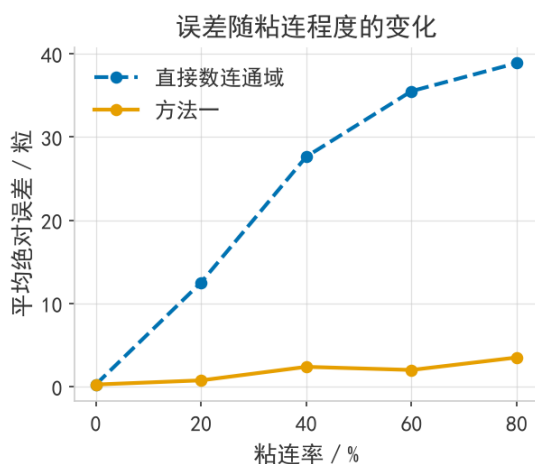
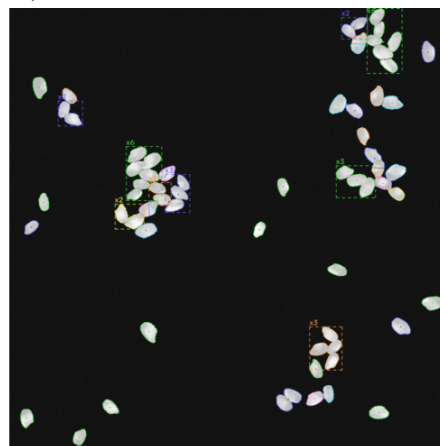


图 1：上排为方法一在真实照片与重度粘连合成图上的逐粒计数结果；下排左为计数误差随粘连率的变化，右为四种做法在同一批测试图上的平均绝对误差。

目 录

1 引言	3
2 相关工作	3
3 方法一：自己量尺度、自己切开（几何方法）	3
3.1 几个基本概念	3
3.2 二值化：让程序自己挑	4
3.3 尺度自标定：量出一粒米有多大	5
3.4 判断是否粘连	5
3.5 放种子并切开粘连块	5
3.6 检查与修正	6
4 方法一的实验与分析	8
4.1 数据集	8
4.2 评价指标	8
4.3 与其他方法比较	8
4.4 误差随粘连程度的变化	9
4.5 图像质量变差时的表现	9
4.6 消融实验	10
4.7 与 SAM 3 对比	11
4.8 算力成本对比	12
4.9 让程序跑得更快：先找瓶颈，再谈 GPU	12
5 方法二：把 SAM 3 会的东西教给小网络（学生模型）	14
5.1 为什么会有第二种方法	14
5.2 怎么教，学生学什么	14
5.3 结果与代价	14
5.4 走过的弯路，以及必须说清的前提	14
6 讨论	15
6.1 试过但没用的办法	15
6.2 不足	15
7 结论	16
A 更多计数结果	16
A.1 D1: 真实照片	17
A.2 D2: 合成粘连图	17
A.3 D3: 低分辨率	18
B 代码、数据与复现方式	19

1 引言

本次作业的题目是：数出照片里有多少粒米，并解决连体米粒计数不准的问题。老师在课上只给出了这一句要求，具体用什么方法、分哪几步，是留给我们自己想的部分。

我先按最基本的思路做了一遍：转灰度，用 Otsu 法^[1]自动选一个阈值把米粒和背景分开（该方法选取使前景与背景的两类间方差最大的灰度值），再统计连通域的个数——连通域指二值图中连在一起的一片前景像素，正常情况下一粒米就是一个连通域。米粒互不接触时这一流程没有问题；一旦两粒米挨在一起，它们在二值图中连成一块，只被计为 1 粒，米粒越密，漏计越多。实测粘连率 80% 时平均少数 38.9 粒（图 6），这正是题目所指的难点。

查阅文献时注意到，这个问题在已发表的工作中也常被回避。Kurade 等人^[2]的米质检测系统采用“腐蚀取标记 + 注水分割”（注水分割即 watershed 算法，常译作“分水岭算法”），但文中承认该算法分不开互相接触的米粒，因而把这些米粒从数据集中剔除。既然课程题目把这一点单独提了出来，本文就试着正面处理它。

本文的工作可以概括为以下几点：

1. 由程序从图像自身量出单粒米的尺寸与形状，后续判据均以此为单位，不写死像素常数；
2. 不预先指定颜色通道与阈值方式，而是把若干组合逐一试过，按一个打分函数自动选取；
3. 以“既大且凹”判断连通域是否为粘连块，再用注水分割切开；
4. 切分后按面积与形状复核，并剔除硬币、木框等非米目标。

在三组性质不同的数据上，上述方法的平均绝对误差为 4.73、1.77 与 1.93 粒，优于本文实现的五种经典对比方法；粘连率 80% 时误差约为直接统计连通域的十一分之一。在此之外，本文还用 SAM 3 的输出作伪标签蒸馏了一个 27 万参数的小网络（第 5 节），其总体误差进一步降到 0.68 粒。代码、数据来源与复现方式见附录 B。

2 相关工作

分开粘连目标大致有两种思路。

一种是从轮廓入手：两粒米的接触处在轮廓上是一个凹点，找到凹点再连线切开。Tan 等人^[3]用这种方法处理粘连稻谷，Avzalov 等人^[4]进一步用椭圆拟合滤除假凹点。本文将后者实现为对比方法 B5。

另一种是从区域入手：先用注水分割^[5]切，再修正结果。Gamarra 等人^[6]先以大小和凸性判断哪些区域需要切分，切开后合并过碎的部分。本文方法属于后一类，结构上与之相近；不过这是方法做完以后查文献才发现的。

3 方法一：自己量尺度、自己切开（几何方法）

这是本文的第一种方法，主线仍是“二值化—形态学—连通域”，只是把其中的参数交给图像自己决定。它并非先设计后实现，而是一次次失败之后补出来的：二值化在真实照片上失效，才明白通道不该由人替程序指定；判据里的常数换一张图就不成立，才明白尺子只能从图像自身量取。整体流程见图 2，左列是对整张图的处理，右列是对每个连通域的判断与计数；图 3 给出各步骤在一张合成图上的中间结果。下面按实际的摸索顺序叙述。

3.1 几个基本概念

距离变换 D ：对每个前景像素，算它到最近背景像素的距离。米粒中间的值大、边缘的值小，像一座山；两粒米粘连的地方会出现一道山谷。

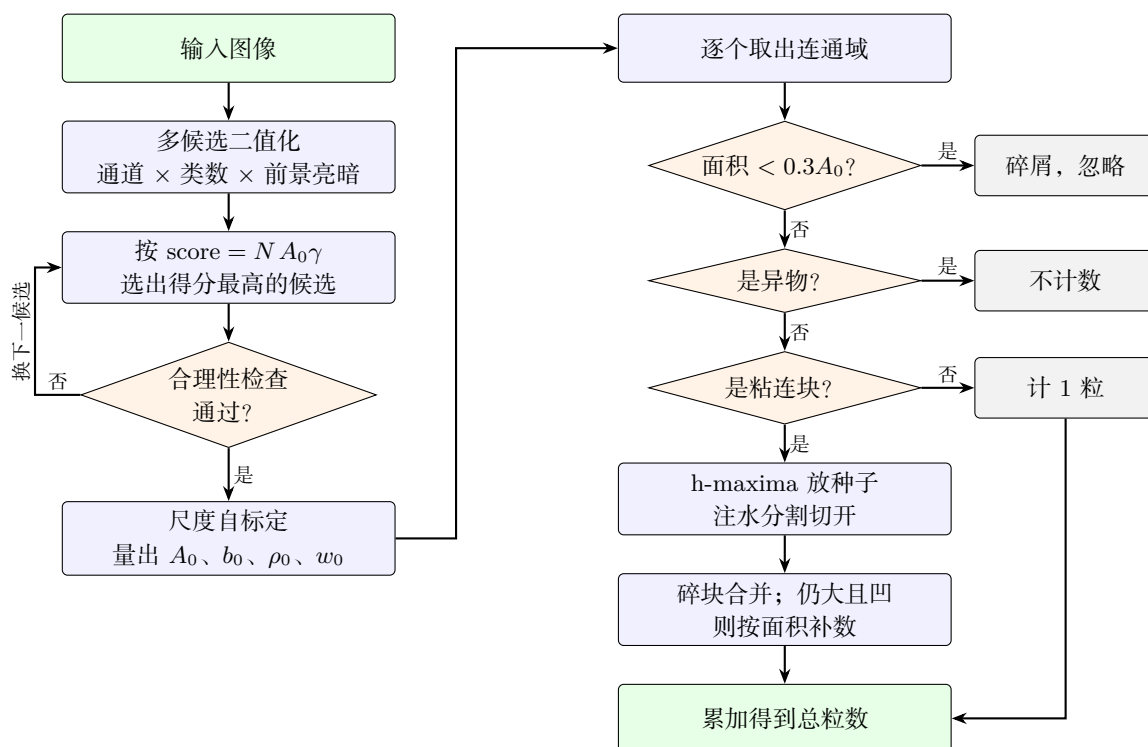


图 2: 方法流程图。

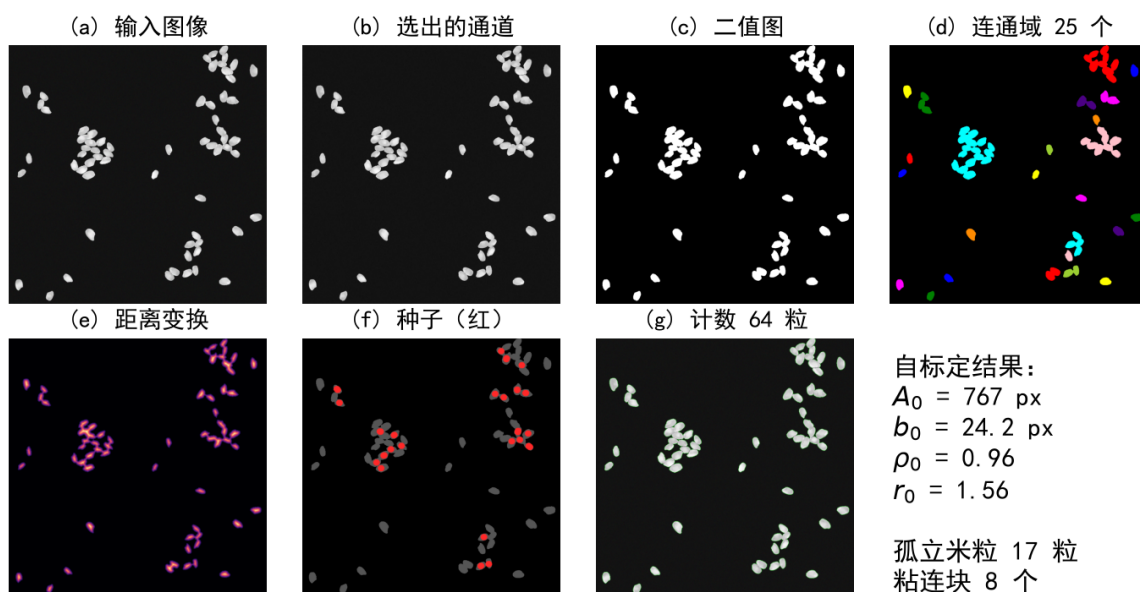


图 3: 各步骤在一张合成图上的中间结果。

注水分割: 把距离图看成地形, 从事先给定的种子处开始“注水”, 两股水相遇的地方就是分界线。给几个种子就切成几块, 所以关键在于种子怎么放。

凸实度 ρ : 区域面积除以其凸包 (包住该区域的最小凸多边形) 的面积。一粒米接近凸形, ρ 接近 1; 两粒粘在一起, 接缝处凹进去, ρ 会变小。

3.2 二值化: 让程序自己挑

起初直接用“灰度 + Otsu”, 在真实照片上便出了问题: 白米放在深色衬布上、衬布又铺在浅色桌面上, 画面有三个亮度层次, 而 Otsu 只分两类, 结果把整块衬布判成前景, 一张图只数出 1 个连通域。

由此可见，通道、类数与前景极性都不宜由人事先指定。本文把这三者展开成候选集合——通道取灰度或 HSV 饱和度，阈值取 2 类或 3 类 Otsu，前景取亮或暗——逐一二值化，再用下式打分择优：

$$\text{score} = N \cdot A_0 \cdot \gamma \quad (1)$$

式中 N 为尺寸接近单粒的连通域个数， A_0 为估计的单粒面积， γ 为前景与紧邻背景的灰度差。 γ 一项是后来补上的：合成图的饱和度通道会被噪声打成上千个小斑点，只算前两项时它们反而得分最高；这些斑点与背景几乎没有反差，计入 γ 后便被排除。

此外还有若干合理性检查，例如米粒长宽比应在 1.25 至 8 之间、单粒面积不超过画幅的 1%。这些检查最初只设了下界，结果 D3 中有 67 张图把木框条纹当成了“米粒”——其长宽比达 25，一条就占画面的 3%。补上上界、并在检查不通过时回退到下一候选之后，这 67 张图的平均误差由 11.45 粒降至 3.57 粒。

3.3 尺度自标定：量出一粒米有多大

后续每一个判断——多大算大、多凹算凹——都需要一个参照物。三组数据的米粒尺寸相差悬殊（短轴自 4 像素至 25 像素），写死一个像素数难以通用，因此这把尺子只能由程序从每张图上量出来。

最初的想法是取面积分布中最小的那个峰作为单粒，实测常常选中比米粒更小的碎屑；改出现次数最多的尺寸，即面积分布的最高峰（代码 1），才稳定下来。面积先取对数，因为粘连块的面积大约是 A_0 、 $2A_0$ 、 $3A_0$ ……，取对数后这些峰间距相等，更容易看清（图 4）。得到 A_0 后，再从同一批接近单粒的连通域里量出短轴 b_0 、凸实度 ρ_0 、宽度 w_0 。

代码 1: 估计单粒面积 A_0

```
1 def estimate_a0(areas):
2     """取对数面积分布的最高峰作为单粒米面积"""
3     log_s = np.log(areas)
4     kde = gaussian_kde(log_s) # 核密度估计，相当于平滑的直方图
5     grid = np.linspace(log_s.min(), log_s.max(), 512)
6     a0 = np.exp(grid[np.argmax(kde(grid))]) # 最高峰对应的面积
7     # 再取 0.65~1.45 倍 A0 之间的连通域求平均，结果更稳
8     band = areas[(areas >= 0.65 * a0) & (areas <= 1.45 * a0)]
9     return band.mean() if len(band) > 0 else a0
```

3.4 判断是否粘连

连通域比单粒米大，未必就是粘连，也可能只是一粒偏大的米。两粒米贴合处一般会形成凹陷，因此本文要求“既大且凹”两个条件同时成立。起初用的是“或”，本想宁可多切，实测在真实照片上误差反而增加 1.7 粒。

米粒很小时（短轴不足 4.5 像素），边缘锯齿明显，凸实度已不可靠。这时改用区域内最大内切圆的直径 w （距离变换最大值的两倍）：两粒米并排，面积翻倍但“厚度”不变，所以用面积和厚度的比值来判断（代码 2），一粒大米这个值约为 1，两粒并排约为 2。

3.5 放种子并切开粘连块

判出粘连块之后，还要决定给注水分割放几个种子。最先想到的是限制种子之间的最小距离，稍作计算便知此路不通：米粒细长（长宽比约 3.7），一粒米内部距离变换的“山脊”比两粒并排时的中心间距还长，最小距离取多少都难以兼顾这两个要求。

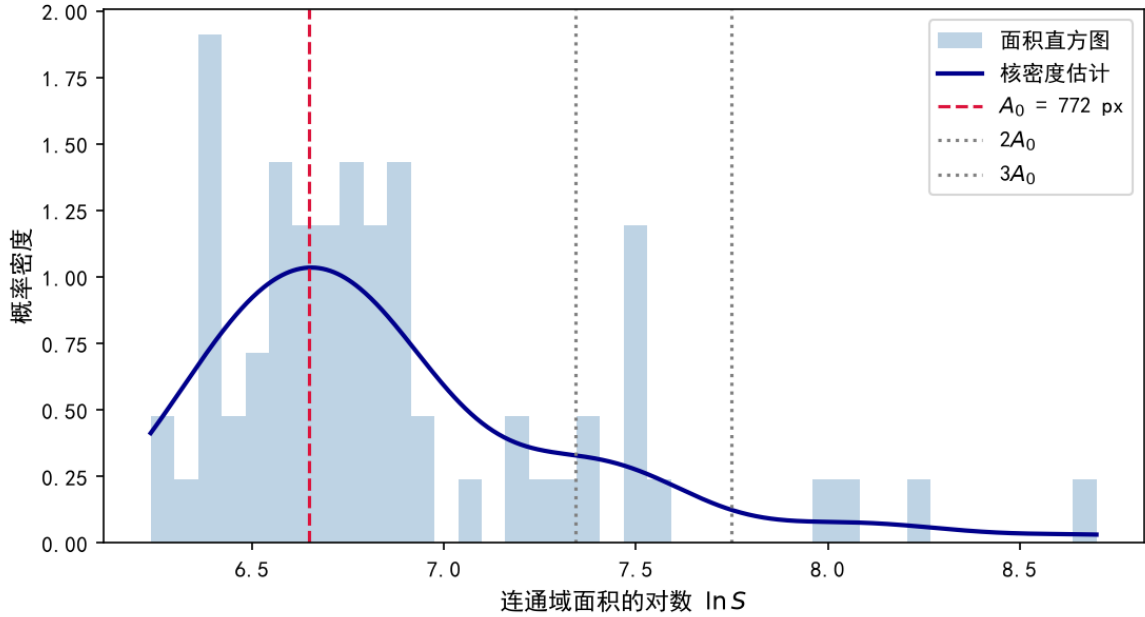


图 4: 面积分布的最高峰即单粒面积 A_0 , 粘连块集中在它的整数倍附近。

代码 2: 判断一个连通域是不是粘连块

```
1 def is_touching(c, cal):
2     """c 是一个连通域, cal 是自标定得到的单粒米参数"""
3     if c.area <= 1.2 * cal.a0:          # 不比一粒米大, 不算粘连
4         return False
5     if cal.b0 < 4.5:                    # 米粒太小, 凸实度量不准, 改看厚度
6         e = (c.area / cal.a0) / (c.width / cal.w0) ** 2
7         return e > 1.5
8     return c.solidity < cal.rho0 - 0.06 # 又大又凹才算粘连
```

换一个角度就有了出路: 两粒米之间是一道深谷, 一粒米内部只是浅浅的起伏, 该看的是深度而非距离。为此对距离变换取 h -maxima, 把高度不足 $h = \beta b_0 / 2$ 的小峰抹平 (代码 3)。此处用短轴 b_0 , 是因为两粒米能靠多近取决于它们的宽度。

代码 3: 放种子并做注水分割

```
1 def split_cluster(mask, cal, beta=0.45):
2     dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
3     h = beta * cal.b0 / 2          # 深度门限跟着米粒宽度变
4     seeds = label(h_maxima(dist, h)) # 只留下足够高的山峰作种子
5     if seeds.max() <= 1:           # 只有一个种子, 不用切
6         return (mask > 0).astype(int)
7     return watershed(-dist, seeds, mask=mask > 0)
```

3.6 检查与修正

注水分割只回答了“可能在哪里切”, 切得对不对还要再检查, 一个粘连块从切开到定出粒数的全过程见图 5。整个计数过程见代码 4: 碎块并入邻居; 切完后仍然又大又凹的块, 说明没切开, 就按面积补数; 面积超过 $3A_0$ 并且特别厚 ($w > 2.5 w_0$) 或者和单粒一样凸的, 判为异物。

代码 4: 计数主循环

```

1 total = 0
2 for c in components:
3     if c.area < 0.3 * a0:                # 碎屑, 忽略
4         continue
5     if is_foreign(c, cal):                # 硬币、木框等, 不计数
6         continue
7     if not is_touching(c, cal):           # 单独的一粒
8         total += 1
9         continue
10    parts = merge_small(split_cluster(c.mask, cal), a0) # 小于 0.4*A0 的碎块并入邻居
11    for p in regionprops(parts):
12        k = p.area / a0
13        if k < 0.4:                        # 没有邻居可并的碎块
14            continue
15        if k > 1.5 and p.solidity < cal.rho0 - 0.03:
16            total += round(k)              # 没切开: 按面积补数
17        else:
18            total += 1

```

异物判据的依据有两条：米粒只是相互挨着、不会融为一体，因此一堆米最厚处仍只有一粒米那么厚；几粒米粘连通常会留下凹口，面积数倍于单粒却与单粒一样凸的区域，多半不是米堆。实测单粒的 w/w_0 介于 0.98 与 1.14 之间，真实粘连块不超过 2.0，而硬币为 8.6、木框条约为 4。该判据最初是照着 D1 中的硬币写的，要求“更凸且更圆”；D3 的木框条比米粒更细长，恰好从“更圆”这一条漏了过去。去掉这一多余条件后，D3 误差由 2.34 粒降至 1.55 粒，D1、D2 则没有变化。

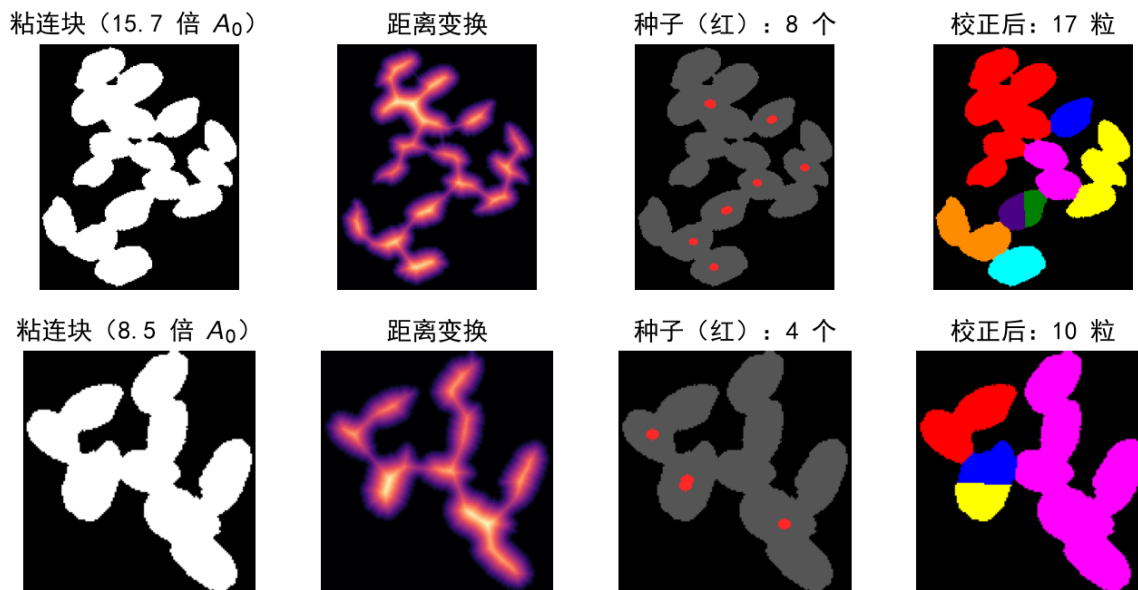


图 5: 粘连块的处理过程，从左到右依次为粘连块、距离变换、种子和最终结果。

4 方法一的实验与分析

4.1 数据集

D1、D3 以及 D2 所用的米粒素材均取自 Roboflow Universe 上的公开数据集（CC BY 4.0 许可），其中 D2 由本文自行合成。

D1（真实照片，51 张）：Roboflow 上的 rice 数据集^[7]，手机拍摄，深色布上摊开的米粒，每张 80–116 粒，带一枚参照硬币，每粒米都有人工标注框。米粒基本不粘连。

D2（自制合成图，40 张）：从 RICE 数据集^[8]中 Karacadag 品种的单粒米照片里抠出米粒，随机旋转、缩放后贴到深色画布上，粘连率分 0%、20%、40%、60%、80% 五档，每档 8 张，每张 44–98 粒。其价值在于真值由构造方式精确给定，且粘连程度可控。

D3（低分辨率，719 张）：Roboflow 上的 RICE 数据集^[9]， 224×224 的小图，每张约 14 粒。该数据集导出时做过翻转增强，同一源图存在多个副本，按源图名去重后保留 719 张。

4.2 评价指标

记第 k 张图的真值为 g_k 、预测为 p_k ，主要指标为平均绝对误差 $\text{MAE} = \frac{1}{n} \sum_k |p_k - g_k|$ ，同时给出准确率 $1 - \text{MAPE}$ （MAPE 为平均相对误差 $\frac{1}{n} \sum_k |p_k - g_k|/g_k$ ）与最差单张误差：平均值小并不排除个别图像错得离谱，两者需要一并看。

4.3 与其他方法比较

为使比较只落在“如何分开粘连”这一步上，所有方法都跑在同一份二值图上：B1 直接统计连通域个数，即不对粘连作任何处理；B2 用前景总面积除以单粒面积；B3 距离变换注水分割，种子取 $D > 0.5 \max D$ （OpenCV 教程的做法）；B4 腐蚀后取连通域作种子，即文献^[2]的做法；B5 轮廓凹点 + 椭圆拟合^[4]。另外还和视觉大模型 SAM 3 做了对比（第 4.7 节）。

结果见表 1，本文方法在三组数据上均优于 B1–B5。其中有三点值得说明。

表 1：三组数据上的总体结果，“最差”为最差单张误差（粒）。B5 和 SAM 3 是在每个数据集上分别调到最好后的结果。

方法	D1 真实稀疏			D2 合成粘连			D3 低分辨率		
	MAE	准确率%	最差	MAE	准确率%	最差	MAE	准确率%	最差
B1 连通域计数	6.31	93.25	19	22.95	67.94	51	3.31	76.17	16
B2 面积估算	22.76	74.72	87	2.30	96.69	18	18.39	-30.64	76
B3 距离变换注水分割	74.90	17.77	123	8.20	88.63	24	8.27	42.82	18
B4 腐蚀标记注水分割 ^[2]	5.45	94.21	26	22.93	67.97	51	4.29	69.37	16
B5 凹点 + 椭圆拟合 ^[4]	5.76	—	—	2.08	—	—	3.32	—	—
本文 ASW-SPC	4.73	95.02	24	1.77	97.46	14	1.93	85.96	17
SAM 3（零样本）	3.61	96.11	24	72.15	0.00	98	0.18	98.39	19

其一，B4 在合成数据上的误差与 B1 逐档相同。起初我以为是代码写错，排查后才明白：一次 3×3 腐蚀并不足以分开粘连块，每块仍只产生一个种子，注水分割实际上什么也没有切开，等于退化成连通域计数。

其二，B3 在真实照片上失效，误差达 74.90 粒。它的阈值取自整张图距离变换的最大值，而画面中的硬币远大于米粒，把阈值整体抬高，绝大多数米粒的种子因此被抹掉。这从反面说明了尺度为什么必须从米粒自身量起。

其三，B5 难以同时兼顾两种尺度。它的凹点检测需要在轮廓上作一个小圆，统计圆内属于前景的比例；米粒仅 4 像素宽时，圆几乎盖住整粒米，孤立单粒也会被判出凹点，而把圆调大之后，它在 D2 上又几乎不再切分。表中给出的已是它在各数据集上分别调到最优的配置。还需说明，原文的椭圆拟合为穷举搜索，本文实现的是贪心近似，这一点对它未必公平。

4.4 误差随粘连程度的变化

表 2 与图 6 直接回答了题目所问的那半句话：粘连率自 0% 升至 80% 的过程中，连通域计数的误差由 0.25 粒一路升到 38.88 粒，本文方法在这五档上均未超过 3.5 粒。逐张的预测值与真值对照见图 7，未见系统性的整体偏移。

表 2: 合成数据上按粘连率分层的平均绝对误差（粒）。

粘连率	B1	B2	B3	B4	B5	本文
0%	0.25	1.12	0.00	0.12	0.00	0.25
20%	12.50	1.12	4.50	12.50	7.62	0.75
40%	27.62	2.25	10.62	27.62	15.88	2.38
60%	35.50	2.50	11.12	35.50	16.38	2.00
80%	38.88	4.50	14.75	38.88	18.25	3.50

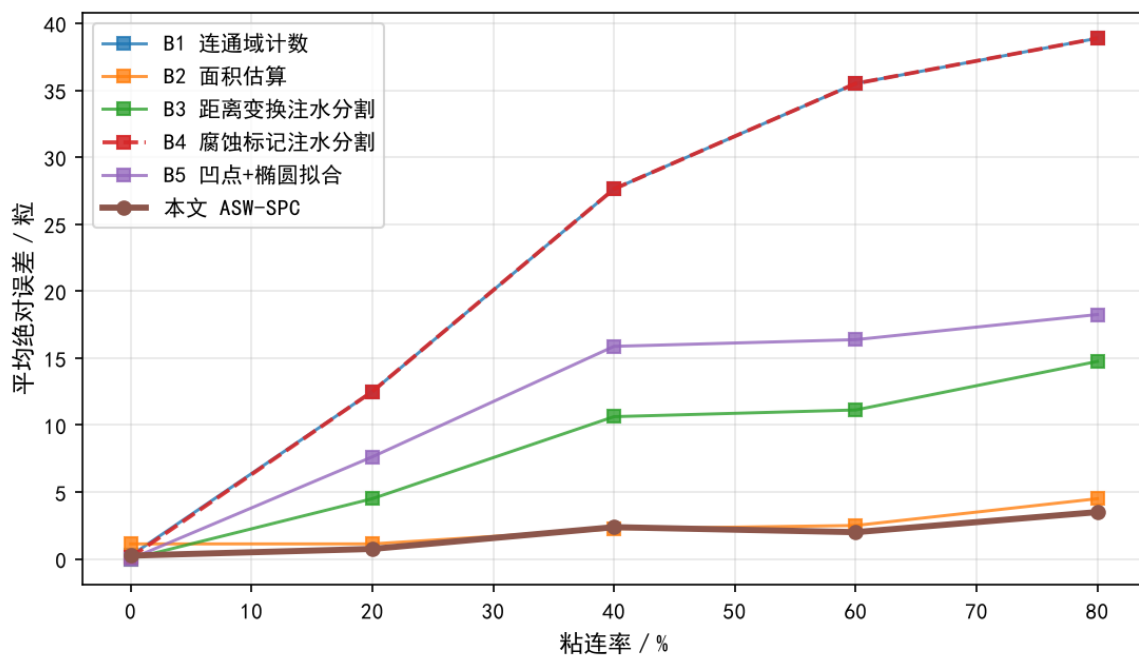


图 6: 计数误差随粘连率的变化。

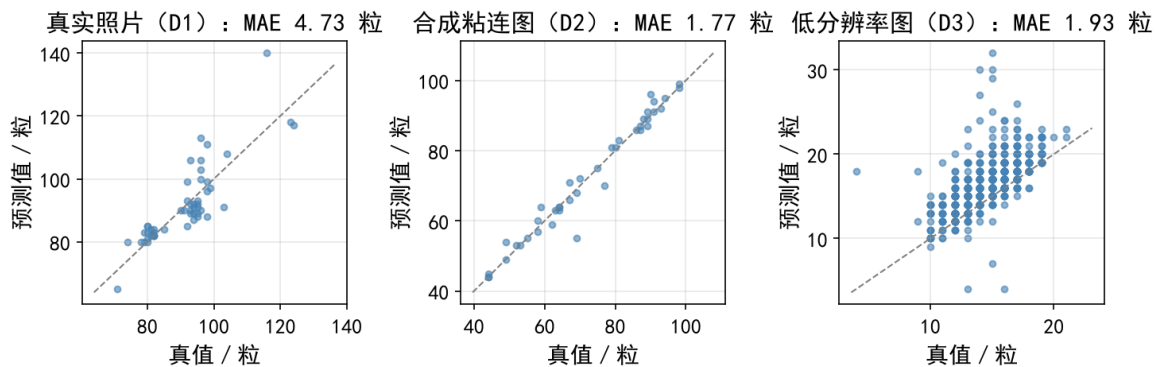


图 7: 预测值与真值对照，虚线表示两者相等。

4.5 图像质量变差时的表现

为了解方法的失效边界，本文在 D2 上人为施加模糊、噪声、对比度压缩与降采样四类退化（表 3，模糊强度以单粒宽度 b_0 为单位）。

噪声、对比度、分辨率三项的影响都不大：噪声加到 45 个灰度级、对比度压到 8%，误差未见上升；降采样至 1/6（米粒只剩约 2 像素宽）时中位误差为 4 粒，属于优雅退化。模糊则不同——中位误差变化不大，平均误差却会突然暴涨，原因是个别图像模糊后二值化整个选错，一张图数出上千粒。正因为两者会给出相反的印象，表中平均值与中位数一并列出。

表 3: 不同退化下的计数误差（D2, 40 张）。“崩溃”指单张误差超过 100 粒的张数。

退化类型	强度	平均误差	中位误差	崩溃场景	B1 平均误差
高斯模糊	0.00 b_0	1.77	1.00	0/40	22.95
	0.20 b_0	251.88	1.00	2/40	156.95
	0.35 b_0	79.28	2.00	2/40	65.78
	0.70 b_0	77.35	13.50	13/40	59.95
加性噪声	0	1.77	1.00	0/40	22.95
	20	1.60	1.00	0/40	23.00
	45	1.57	1.00	0/40	22.95
对比度保留	8%	1.62	1.00	0/40	22.98
	30%	1.85	1.00	0/40	22.95
	100%	1.77	1.00	0/40	22.95
降采样倍率	1.0×	1.77	1.00	0/40	22.95
	3.0×	2.30	1.00	0/40	23.88
	6.0×	5.35	4.00	0/40	26.00

4.6 消融实验

逐项关闭或改动某一步的结果见表 4。

表 4: 去掉或改动某一步后的平均绝对误差（粒），第一行为完整方法。

消融项	D1	D2	D3
完整方法	4.73	1.77	1.93
去掉尺度自标定（ A_0 取数据集均值）	8.57	0.90	1.91
去掉形状先验校正	5.04	11.68	1.66
去掉异物剔除	5.02	1.77	3.78
异物剔除去掉宽度判据	4.82	1.77	2.06
异物判据附加“更圆”条件（最初写法）	4.84	1.77	3.01
粘连判据改为“或”	6.43	3.00	1.55
仅灰度通道	4.41	1.77	—
加 3×3 中值滤波	7.27	1.70	—

其中比较明显的几点：

- 去掉“按面积补数”等修正后，D2 误差自 1.77 粒升至 11.68 粒。重度粘连时注水分割往往只切开一部分，其余粒数靠面积推断补齐；这一步的贡献比我原先设想的主角——自适应种子——还要大。
- 去掉异物剔除后，D3 误差自 1.93 粒升至 3.78 粒，多出来的主要是被当成米的木框。
- 去掉尺度自标定后，D1 误差自 4.73 粒升至 8.57 粒；D2 反而略有改善，因为合成图的米粒尺寸整齐，用全局固定值比逐图现算更稳。这说明该步的价值体现在真实数据的尺寸参差上。
- 仅用灰度通道时 D1 略好，D3 却离不开饱和度通道，故保留完整的候选集合。

β 取 0.45 的依据只在 D2；D1 上 β 从 0.45 到 1.0 结果相同，没有区分度。

4.7 与 SAM 3 对比

SAM 3^[10] 是一个以文字提示驱动的开词表分割模型——所谓开词表，是指它不限于训练时见过的固定类别，给一个名词即可分割对应的目标，给出“rice grain”之类的提示即可返回每粒米的掩膜，无需针对本任务训练。本文先在 D1 上比较了不同提示词与阈值（表 5）。需要如实说明的是，提示词选得恰当时，SAM 3 比本文方法更准（表 6，white seed 在三组数据上分别为 3.29、0.65、0.07 粒）。但它对提示词相当敏感：五个提示词中有三个在合成图上一粒也没检出，其中包括看起来最自然的 rice grain，而模型并不会提示自己失败。要知道哪个提示词可用，仍然得拿带标注的数据去试。

表 5: SAM 3 在真实照片（D1）上使用不同提示词和阈值的平均绝对误差（粒）。

文本提示	$\tau = 0.15$	$\tau = 0.20$	$\tau = 0.25$	$\tau = 0.30$	$\tau = 0.35$	$\tau = 0.40$
grain	18.86	9.04	4.69	3.63	3.61	4.53
rice grain	8.71	6.41	8.12	13.45	28.41	51.78
white rice grain	18.10	10.00	7.16	5.92	5.88	6.71

表 6: SAM 3 各提示词在各数据集上分别调到最好时的误差， τ^* 为对应阈值。

文本提示	D1		D2		D3	
	MAE	τ^*	MAE	τ^*	MAE	τ^*
rice grain	6.41	0.20	72.15	0.05	0.62	0.20
grain	3.61	0.35	72.15	0.05	0.07	0.35
white rice grain	5.88	0.35	72.12	0.05	0.08	0.20
white seed	3.29	0.40	0.65	0.40	0.07	0.35
seed	24.45	0.25	27.45	0.20	0.08	0.40

为便于逐粒核对而非只看数字，图 8 至图 10 给出每组数据各一个场景的结果。画法是只勾轮廓、不填充，相邻实例着以不同颜色；虚线框与 $\times N$ 标出的区域并未被真正切开，其粒数由面积推断而来；灰色并标 $\times$ 者为判定的异物，不计入总数。

真实照片（D1） 本文方法：99 粒（真值 98 粒） 真实照片（D1） SAM 3：93 粒（真值 98 粒）



图 8: 真实照片（D1）上的结果，左为本文方法、右为 SAM 3。

D1 中两种方法都未把硬币计入；D2 中两者均与真值一致，本文方法有数处是靠面积补齐的；D3 中木框亮边被正确剔除，但漏掉了一粒颜色偏暗的米。

合成粘连图 (D2) 本文方法: 64 粒 (真值 64 粒) 合成粘连图 (D2) SAM 3: 64 粒 (真值 64 粒)

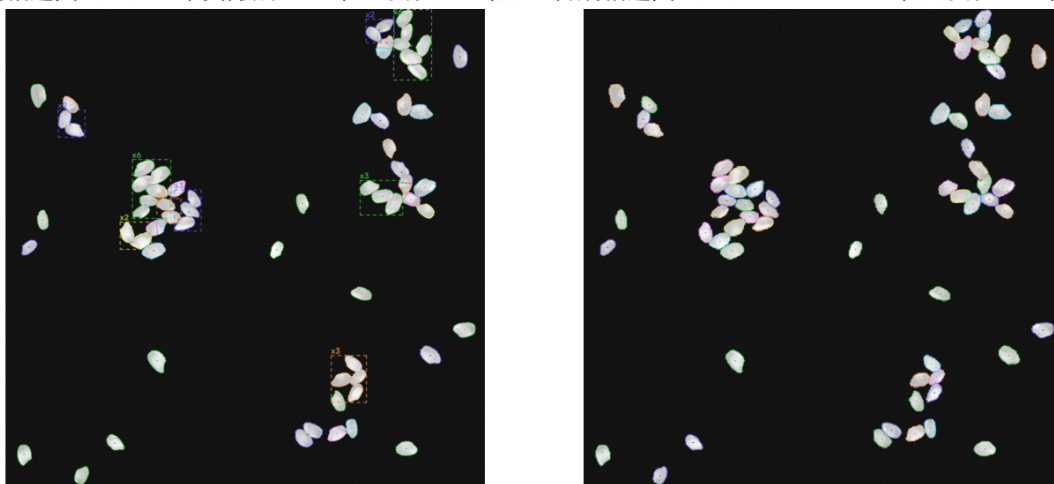


图 9: 合成粘连图 (D2) 重度粘连场景上的结果, 左为本文方法、右为 SAM 3。

低分辨率图 (D3) 本文方法: 14 粒 (真值 15 粒) 低分辨率图 (D3) SAM 3: 15 粒 (真值 15 粒)

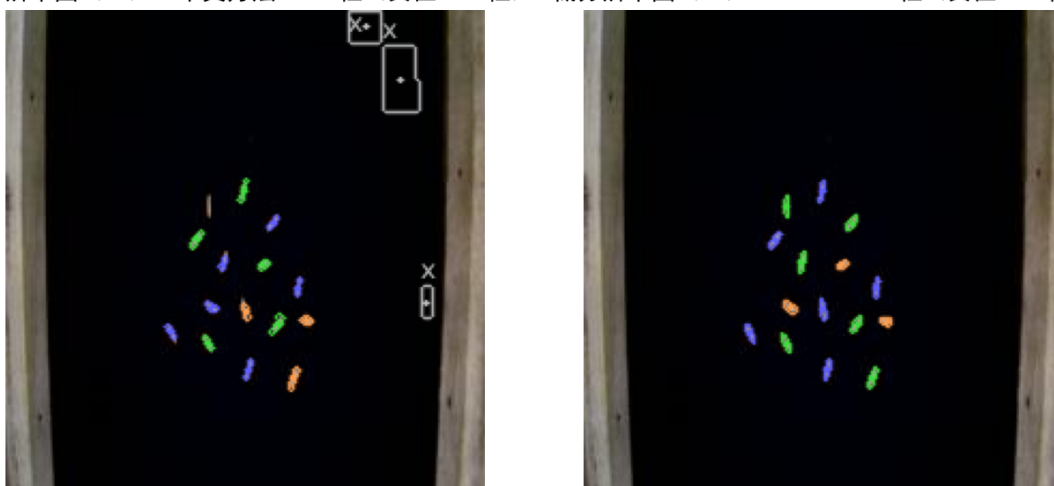


图 10: 低分辨率图 (D3) 上的结果, 左为本文方法、右为 SAM 3。

4.8 算力成本对比

除精度之外, 本文也比较了两种方法的算力开销。在同一台笔记本 (Intel i9-13900H, RTX 4080 Laptop GPU, 12 GB 显存) 上, 每组数据取 10 张图, 测每张的处理时间 (不含读图), 并记录峰值内存和显存, 结果见表 7。SAM 3 在 CPU 上太慢, 每组只测了 1 张; CPU 上用 32 位浮点, GPU 上用 16 位浮点 (bfloat16)。

本文方法的耗时是按第 4.9 节优化后测的; 为了公平比较, 那一节也给出了它的 GPU 版本。

有显卡时 SAM 3 每张约 0.21 s, 且与图像大小基本无关; 本文方法的 GPU 版在 D1 上更快 (0.15 s), 在 D2 上略慢。但 SAM 3 需先加载 8.4 亿参数、3.2 GB 权重, 运行时占用 5.6 GB 内存与 2.5 GB 显存; 本文方法没有需要学习的参数, GPU 版仅占 0.09 GB 显存, 相差约 28 倍。没有显卡时优劣反转: SAM 3 每张耗时 17 s 以上, 本文方法在 CPU 上为 0.03 至 0.63 s; 在低分辨率的 D3 上, 本文方法仅用 CPU 也比 SAM 3 用显卡快约 6 倍。

4.9 让程序跑得更快: 先找瓶颈, 再谈 GPU

最初 D2 每张需 6.6 s, 我的第一反应是“慢就上 GPU”。先做了一次性能剖析 (profile, 统计程序在各个函数上花掉的时间), 结果与设想相去甚远: 逐像素运算 (Otsu 0.6 ms、连通域标

表 7: 单张图的处理时间和资源占用。显存为分配的峰值; SAM 3 (CPU) 每组只测 1 张。

项目	本文 (CPU)	本文 (GPU)	学生 (GPU)	SAM 3 (GPU)	SAM 3 (CPU)
D1 每张耗时 / s	0.37	0.15	0.014	0.21	17.7
D2 每张耗时 / s	0.63	0.40	0.025	0.21	17.3
D3 每张耗时 / s	0.033	0.071	0.002	0.21	18.3
模型参数量	0	0	0.27 M	840 M	840 M
权重文件 / MB	0	0	1	3281	3281
模型加载 / s	—	—	0.1	3.1	2.5
峰值内存 / GB	0.30	0.70	1.17	5.61	5.07
峰值显存 / GB	—	0.10	0.23	2.50	—

记 2.0 ms、距离变换 3.2 ms) 8 个候选全算一遍也只有约 50 ms, 不到总时间的 1%; **98% 花在逐个连通域算凸包和长短轴上**——Python 里的逐对象循环, 搬到 GPU 上并不会变快。原因在于一张 D2 图的 8 个候选共约 18300 个连通域, 其中真正可能是米粒的只有灰度通道的约 180 个, 其余皆为饱和度通道的噪点, 而程序对它们也逐个算了凸包。

这三步能想到, 和上学期的两门课有直接关系。操作系统原理讲进程与线程调度时说过: 线程不是越多越好, 线程数超过核数之后, 时间会花在上下文切换和争抢上, 反而更慢。所以当我看到“一张图占了 17 个核却还很慢”时, 第一反应就是去查线程数, 而不是去买更好的显卡; 多进程并行为什么一开始没有效果, 也是同一个道理——每个进程内部又各自开了 20 个线程, 互相抢核。计算机组成原理讲的则是另一半: 先看程序到底把时间花在哪一类操作上, 再决定优化什么; 以及并行部件都有启动开销, 粒度太小时并行反而不划算。GPU 在 224×224 的小图上比 CPU 还慢, 正是核函数启动开销大于计算本身的例子。这两门课给我的共同提示是: **先测量, 再动手**。

第一步: 不算用不到的东西。面积和外接框用 OpenCV 的连通域统计一次取回; 长短轴改用二阶矩批量算出 (与逐对象结果的差别在 10^{-11} 量级); 凸包只给面积大于 $0.3A_0$ 的区域算——更小的会被当作碎屑丢掉, 不会被问到凸实度。D2 由此降到 2.7 s。

第二步: 把线程数限制为 1。这一步颇违直觉。我本以为程序是单线程的, 实测却发现处理一张图占满了 17 个核——标定中有大量很小的矩阵运算, OpenBLAS 默认开 20 个线程去应付, 调度开销远大于计算本身。锁成单线程后, 单张反而更快: D1 由 693 ms 降至 431 ms, D3 由 82 ms 降至 45 ms。这也解释了多进程起初几乎没有加速的原因——各进程争抢的是同一批核。线程锁定之后再按图并行, D3 用 20 个进程处理 240 张仅需 2.0 s, 合每张 8.4 ms, 相对单进程提升 4.4 倍。

第三步: 调整判断顺序, 使绝大多数凸包不必计算。这是收益最大的一步, 改动却只是把代码的次序换了一下。候选的得分 $NA_0\gamma$ 并不含凸实度, 真正用到它的只有“这批区域像不像米”那道合理性检查。原先的写法是先检查、再排序, 于是每个候选都得先把凸包算出来; 改为先按得分排序、再自高而低逐个检查后, 选出的候选相同——都是通过检查的最高分者——但排在其后的候选一次凸包也不必算。噪点候选的对比度 γ 很低, 本就排不到前面, 那 18000 多次凸包计算于是自行消失。D2 由 2.7 s 降至 0.63 s, D1 降至 0.37 s, D3 降至 0.033 s。

以上三步在三组共 810 张图上逐张比对, 计数与优化前一致, 表 1 中的各项指标也未发生变化。

第四步: 把逐像素的部分放到 GPU 上。通道转换、Otsu、开闭运算、连通域标记、距离变换改用 CuPy 和 cuCIM; 注水分割仍留在 CPU 上, 因为 cuCIM 没有 GPU 版本, 而它只在很小的粘连块截图上运行, 本来就不占时间。结果见表 7: GPU 版在 D1、D2 上分别为 0.15 s 和 0.40 s, 但在 D3 上反而比 CPU 慢一倍 (0.07 s 对 0.03 s)——D3 的图只有 224×224 , GPU 启动核函数的开销比算完整张图还多。

精度上 GPU 版与 CPU 版接近, 但并不逐张一致, 差异来自两处实现细节: OpenCV 的距离变换是近似算法, GPU 版用的是精确欧氏距离变换; 饱和度通道在 GPU 上按同样公式重算, 取整与 OpenCV 略有出入。正文结果仍以 CPU 版为准。

回头看, 收益最大的三步——少算用不到的区域、把线程锁成 1、调整判断顺序——都不需

要显卡，所得计数也与原来一致；最先想到的 GPU 反倒是收益最小、代价最大的一步。整条流水线合计快约 10 倍（D2 由 6.6 s 降至 0.63 s）。先测量、再优化，这是我这次最实在的收获，也是上述两门课留下的最有用的一条经验。

5 方法二：把 SAM 3 会的东西教给小网络（学生模型）

5.1 为什么会有第二种方法

第二种方法是做完上一节的实验之后才想到的。那里留下一个我不太满意的结果：本文方法在 D3 上无论怎么调都与 SAM 3 差距明显，且失败的样子大多属于同一类——木框亮边被当成米，颜色偏暗的米被漏掉。追究下去，几何判据只知道米长什么样，并不知道什么是米（第 6.2 节）。既然 SAM 3 知道，能否把它知道的东西教给一个小模型？本节所做的就是这件事。

5.2 怎么教，学生学什么

标签怎么来。用 SAM 3 处理每张照片，取其每个掩膜的质心，得到一串点坐标作为伪标签；合成图 D2 则直接采用生成时记录的米粒中心，不必经过教师。人工标注全程不参与训练，只用于测试。

学生学什么。学的不是分割，而是密度图：在每个米粒处放一个高斯，整张图的积分即为粒数。这样安排有两点考虑。其一，米粒只有 4 像素宽，要求网络给出清晰的实例边界并不现实，让它在米粒处“鼓起一个包”却是可行的；其二，损失函数不依赖于把相邻两粒分开，而那恰是几何方法最吃力之处。高斯的宽度按每张图自身的米粒间距确定，因此 4 像素与 25 像素的米粒在网络看来尺度相当。

网络与训练。三层 U-Net，27 万参数，权重文件 1 MB。按 7:1:2 划分训练、验证与测试集，划分以源图为单位，以免同一张照片的增强副本跨集泄漏。在 RTX 4080 上训练 6000 步，约 6 分钟。

表 8：学生模型与本文方法、教师在同一批测试图上的平均绝对误差（粒）。真值为人工标注（D1、D3）或合成真值（D2），三者都未用于学生的训练。

数据集	张数	直接数连通域	方法一	方法二：学生模型	SAM 3（教师）
D1 真实照片	10	6.20	4.90	3.91	2.40
D2 合成粘连	8	24.38	1.25	1.29	1.00
D3 低分辨率	144	3.23	1.78	0.42	0.06
合计	162	4.46	1.95	0.68	0.25

5.3 结果与代价

结果见表 8。为便于比较，表中一并列出了不作任何粘连处理时直接统计连通域的结果（4.46 粒），四者测的是同一批图。学生模型的总体误差为 0.68 粒，优于方法一的 1.95 粒；在训练数据最多的 D3 上差距最明显（0.42 对 1.78），D1 上亦更好（3.91 对 4.90），D2 上两者相当。它仍不及教师的 0.25 粒，这符合预期，蒸馏一般不会超过教师。但它只有 27 万参数、1 MB 权重，约为 SAM 3 的三千分之一，每张图耗时 2 至 25 ms，比 SAM 3 快 8 到 100 倍（表 7）；更要紧的是不必挑选提示词——SAM 3 换一个提示词就可能一粒也检不出，学生则把选定的那套行为固化了下来。

5.4 走过的弯路，以及必须说清的前提

第一次训练出来的学生总体误差为 3.31 粒，反而不如几何方法。排查出两个原因：一是训练步数不足，训练集上的误差与测试集相当，属于欠拟合；二是采样不均衡——训练集 567 张中

D3 占了 503 张，按图随机取样时约有 89% 的时间花在 D3 上，米粒最大的两组几乎没被学到。改为先随机选数据集、再从中取图之后，D1 由 8.21 粒降至 3.91 粒，D2 由 9.03 粒降至 1.29 粒。

还有一个前提必须说清楚：学生的成绩建立在教师标签的质量之上，而这一点是实测过的。与人工标注相比，教师给出的点在 D3 上准确率 0.993、召回率 1.000、计数误差 0.10 粒，在 D1 上依次为 0.957、0.967 与 3.29 粒；在这两组数据上，SAM 3 的伪标签确实足够好。但选出这套提示词本身用到了带标注的数据，因此严格地说，学生模型省掉的是推理时的大模型，而不是标注本身。

6 讨论

6.1 试过但没用的办法

上采样。真实照片中的米粒只有 4 至 6 像素宽，我曾设想先把粘连块放大再算距离变换。实测未见改善——插值并不会凭空增加原图中不存在的信息。相关代码已删除。

中值滤波。为压住饱和度通道的噪点，我曾加入 3×3 中值滤波，当时确有效果。但式 (1) 补上 γ 之后，噪点已能被排除，两者功能重复，而中值滤波还会把本就只有几像素宽的米粒磨掉一圈。去掉它后 D1 误差由 7.27 粒降至 4.73 粒。由此得到的经验是：加入新机制之后，应当回头检查旧的补丁是否已成多余。

按 D3 调碎屑门限。调高碎屑门限能让 D3 明显变好，D1 却随之变差。查下去才发现，D1 中那些“又小又圆”的区域其实是真米粒——4 像素尺度下被阈值削去一截的米粒本就长这样。这等于针对单个数据集调参，故未采用。

6.2 不足

会被“更像米粒的东西”骗过。我还找过另一组真实照片^[11]想补充粘连数据，结果方法在很多图上选错了前景：它选中的是画面边上直尺的刻度，而不是米粒（图 11）。刻度大小一致、细长、对比强烈，比颜色浅淡、大小不一的米粒更符合式 (1) 的要求。这说明该准则度量的是“像不像一片重复的小目标”，而不是“是不是米”：它只有关于形态的先验，不含关于米的语义，因此形态上更规整的重复结构就可能胜出。此外，这组照片中相当一部分是多个稻米品种同框对照，粒径相差很大，也不满足“同一张图内米粒尺寸相近”这一前提，故最终未用作测试集。

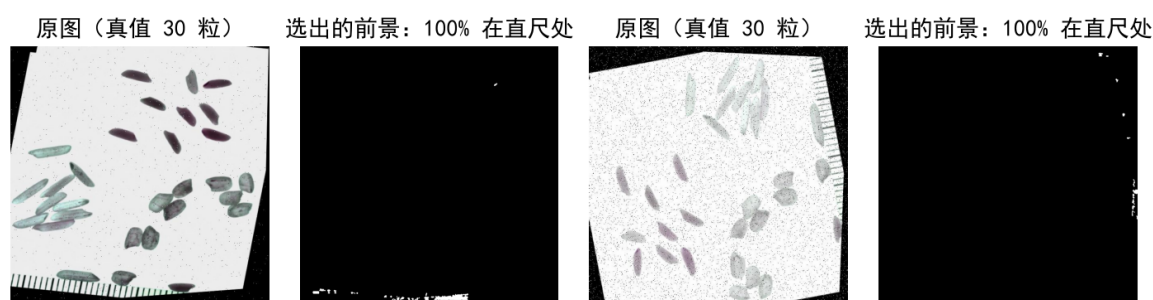


图 11: 方法被直尺刻度误导的两个例子，每例左为原图、右为选出的前景。

模糊时可能选错二值化。第 4.5 节中崩溃的那几张图，是因为模糊后的饱和度通道被阈值化成大量碎点，碎点数目极多，使得式 (1) 的得分反而略高于正确的候选。此问题尚未解决。

D3 存在每图约 1 粒的假阳性。其来源主要是木框边缘的亮块，而非过度切分。

合成数据会美化结果。D2 背景干净、米粒尺寸一致、光照均匀，连“总面积除以单粒面积”这样简单的办法都能做到 2.30 粒，而同一办法在真实照片上的误差是 22.76 粒。看合成数据的结论时须记得这一层。

7 结论

本文实现了一种基于注水分割的粘连米粒计数方法：先由图像自身量出单粒米的尺度，据此判断粘连、控制种子数量，最后按面积与形状复核切分结果。三组数据上的平均绝对误差分别为 4.73、1.77 与 1.93 粒，优于五种对比方法；粘连率 80% 时误差约为直接统计连通域的十一分之一。

做完这份作业，体会最深的一点是：常见做法里有许多写死的数字，例如阈值取 $0.5 \max D$ 、结构元取 3×3 、凹点圆盘取固定半径，它们换一张图就可能失效。B3 被一枚硬币拖垮、B5 在 4 像素米粒上整体失效，都是这个原因。把这些数字换成从图像自身量出的量，方法才真正稳得住。

就精度而言，本文方法不及提示词选得恰当的 SAM 3；但它不需要标注和显卡，内存仅需 0.31 GB，在没有显卡的机器上比 SAM 3 快得多（表 7），且每一步的判断依据都说得清楚，出错时能查到原因。在此基础上，本文又用 SAM 3 的输出蒸馏了一个 27 万参数的学生模型（第 5 节），其总体误差低于几何方法，且无须挑选提示词——这说明“不知道什么是米”这一缺陷，可以通过把大模型的语义知识搬进小网络来弥补。

回到题目本身：“数出有多少粒米”与“解决连体米粒计数不准”这两问，前者在三组数据上的准确率为 95.02%、97.46% 与 85.96%，后者由图 6 作答——粘连率升到 80% 时，直接统计连通域的误差为 38.88 粒，本文方法为 3.50 粒。尚未解决的问题也列在第 6.2 节中，其中模糊条件下二值化选错一项，我目前还没有找到不误伤小米粒的判据；这一点以及全文中判断不当之处，还请老师指正。

参考文献

- [1] Otsu N. A threshold selection method from gray-level histograms[J]. IEEE Transactions on Systems, Man, and Cybernetics, 1979, 9 (1) : 62-66.
- [2] Kurade C, Meenu M, Kalra S, et al. An automated image processing module for quality evaluation of milled rice[J/OL]. Foods, 2023, 12 (6) : 1273. DOI:10.3390/foods12061273.
- [3] Tan Suiyan, Ma Xu, Mai Zhijie, et al. Segmentation and counting algorithm for touching hybrid rice grains[J/OL]. Computers and Electronics in Agriculture, 2019, 162: 493-504. DOI:10.1016/j.compag.2019.04.030.
- [4] Avzalov D R, Komyshev E G, Afonnikov D A. Counting touching wheat grains in images based on elliptical approximation[J/OL]. Vavilovskii Zhurnal Genetiki i Seleksii, 2025, 29 (4) : 608-614. DOI:10.18699/vjgb-25-64.
- [5] Vincent L, Soille P. Watersheds in digital spaces: an efficient algorithm based on immersion simulations[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1991, 13 (6) : 583-598.
- [6] Gamarra M, Zurek E, Escalante H J, et al. Split and merge watershed: a two-step method for cell segmentation in fluorescence microscopy images [J/OL]. Biomedical Signal Processing and Control, 2019, 53: 101575. DOI:10.1016/j.bspc.2019.101575.
- [7] annotation-gnc0j. rice (v1) [DS/OL]. Roboflow Universe (2024) [2025-01-09]. <https://universe.roboflow.com/annotation-gnc0j/rice-nqqbr>.
- [8] thanaree. RICE (v2) [DS/OL]. Roboflow Universe (2023) [2023-12-16]. <https://universe.roboflow.com/thanaree/rice-fgvkm>.
- [9] ezeziel-se47x. RICE (v3) [DS/OL]. Roboflow Universe (2024) [2024-11-17]. <https://universe.roboflow.com/ezeziel-se47x/rice-ljhi4>.
- [10] Meta AI. SAM 3: segment anything with concepts[EB/OL]. (2025) [2026-09-18]. <https://huggingface.co/facebook/sam3>.
- [11] asia-pacific-university-xxaah. rice count (v2) [DS/OL]. Roboflow Universe (2023) [2024-07-12]. <https://universe.roboflow.com/asia-pacific-university-xxaah/rice-count>.

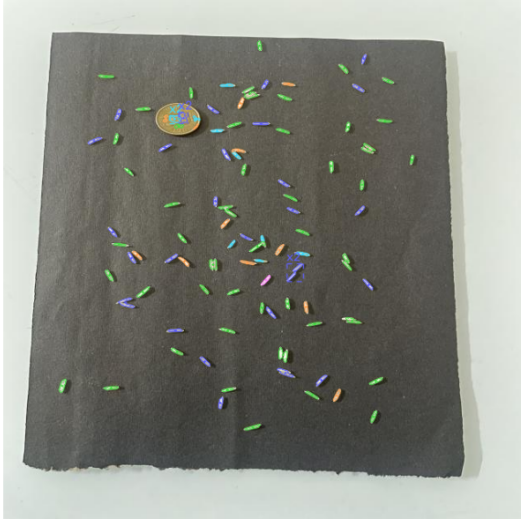
A 更多计数结果

每组数据按顺序等间隔取两个场景，而非择优挑选，因此呈现的是方法的一般表现。左为本文方法、右为 SAM 3，画法与正文图 8 相同。

A.1 D1: 真实照片

两个场景（图 12、图 13）本文方法分别数出 99 粒（真值 98 粒）和 82 粒（真值 82 粒）。硬币在两种方法中都未被计入，但途径不同：本文方法靠第 3.6 节的异物判据将其剔除，SAM 3 则是本就知道硬币不是米。第一个场景多出的那一粒来自硬币边缘的一小块反光，其面积与形状都接近单粒米，几何判据没有理由拒绝它。

本文方法：99 粒（真值 98 粒）



SAM 3：93 粒（真值 98 粒）



图 12: 真实照片（D1）场景一，左为本文方法、右为 SAM 3。

本文方法：82 粒（真值 82 粒）



SAM 3：84 粒（真值 82 粒）



图 13: 真实照片（D1）场景二，左为本文方法、右为 SAM 3。

A.2 D2: 合成粘连图

第一个场景（图 14）没有粘连，两种方法都数出 91 粒，和真值一致。第二个场景粘连率 40%，真值 93 粒，本文方法 92 粒、SAM 3 94 粒。图 15 中可以看到十余个虚线框，说明相当一部分粘连块的粒数是按面积补出的，并非真正切开。

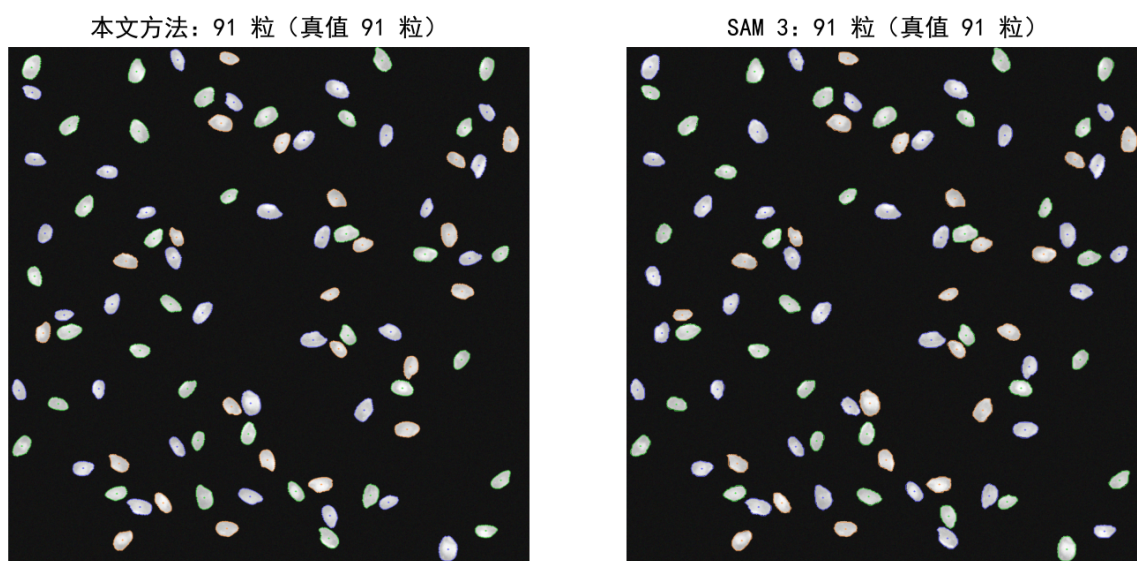


图 14: 合成粘连图 (D2) 中不粘连的场景, 左为本文方法、右为 SAM 3。

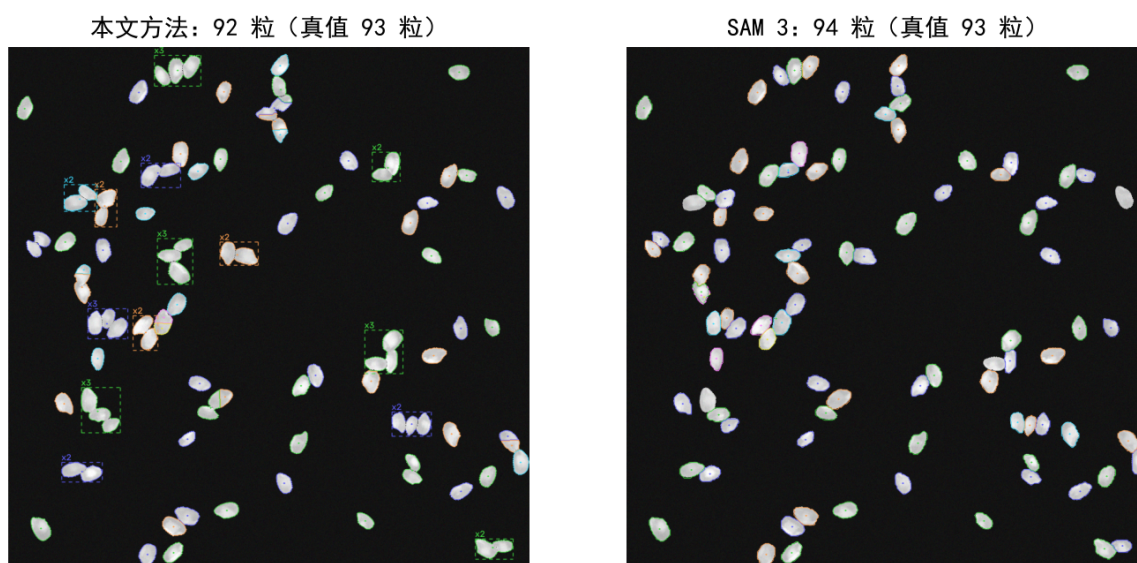


图 15: 合成粘连图 (D2) 中粘连率 40% 的场景, 左为本文方法、右为 SAM 3。

A.3 D3: 低分辨率

两个场景（图 16、图 17）真值都是 15 粒，本文方法分别数出 14 和 16 粒，SAM 3 的轮廓几乎贴合每一粒米。少数的那一粒颜色偏暗，二值化时没分出来；多数的那一个来自本框边缘的亮块。米粒只有 4 像素宽时，形状所能提供的信息已经很少，这也是本文方法与 SAM 3 差距最大的一组。

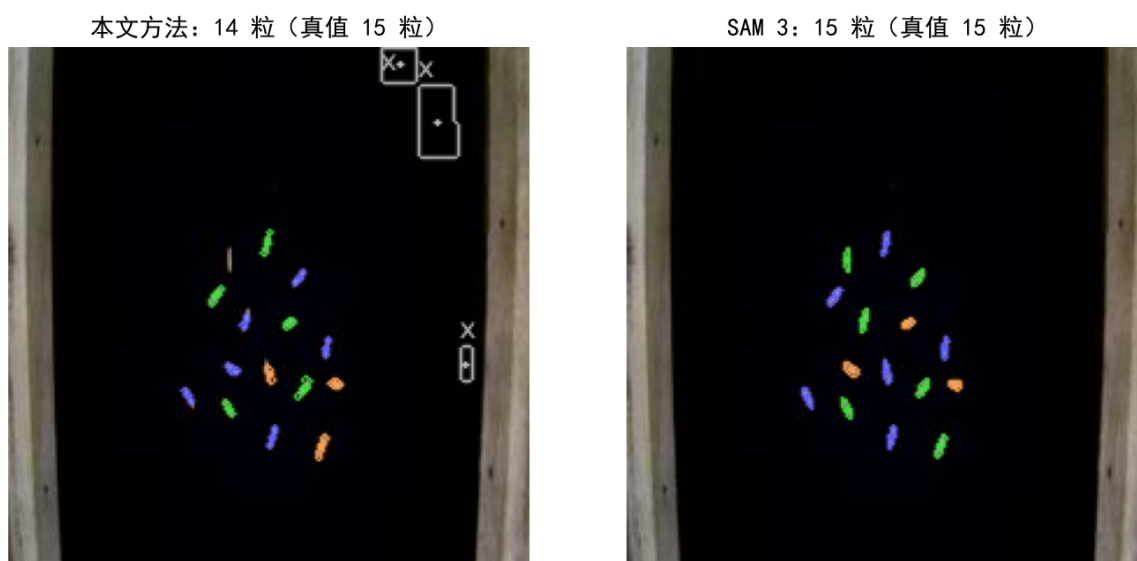


图 16: 低分辨率图 (D3) 场景一, 左为本文方法、右为 SAM 3。

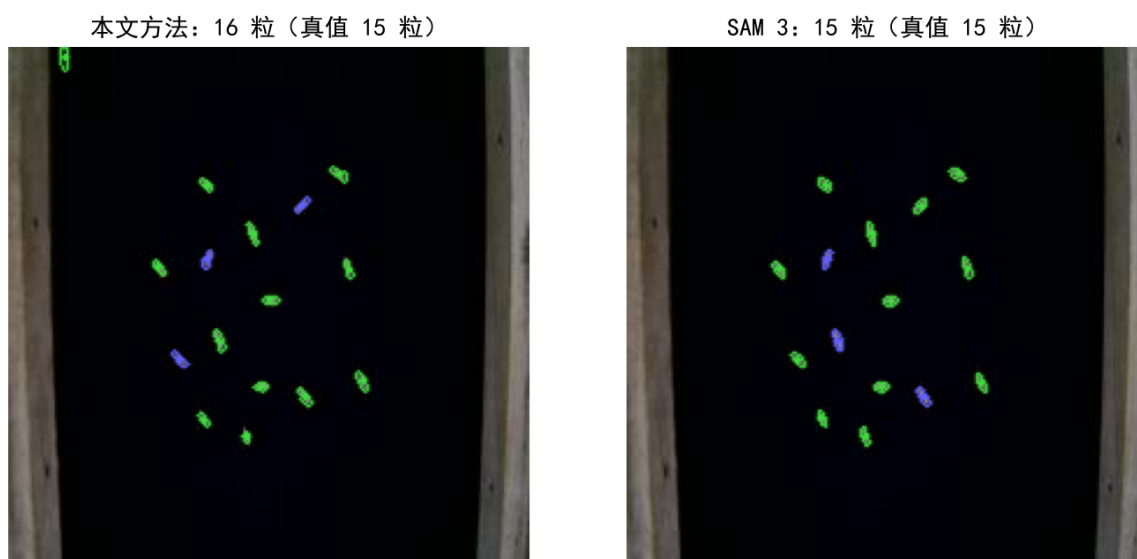


图 17: 低分辨率图 (D3) 场景二, 左为本文方法、右为 SAM 3。

B 代码、数据与复现方式

这一节交代程序与数据的来源, 便于老师核查。

数据。D1、D3 以及 D2 所用的单粒米素材均取自 Roboflow Universe 上的公开数据集 (CC BY 4.0 许可)^[7-9]; D2 由 `src/synth.py` 按固定随机种子生成, 重跑可得到相同的 40 张图与真值。第 6.2 节提到的直尺照片来自文献^[11], 未用作测试集。

代码与复现。环境为 Python 3.13, 依赖 OpenCV、scikit-image 与 SciPy, 学生模型与 SAM 3 另需 PyTorch。各步骤分别实现于 `preprocess.py` (二值化与候选选择)、`calibrate.py` (尺度自标定)、`segment.py` 与 `correct.py` (切分与复核)、`counter.py` (主流程)、`baselines.py` (五种对比方法)。运行 `python -m src.evaluate,src.ablation,src.robustness,src.cost,src.student --eval` 可分别重新生成正文各表, 表中数字均由脚本直接产出, 未经手工修改。